GUÍA PRÁCTICA

Cómo montar un AI Gateway

Con LiteLLM y OmniRoute

Instalación, configuración y uso paso a paso: un único endpoint para tus aplicaciones y varios proveedores de modelos detrás.



@chanti.dev · Octubre 2026

Basada en la documentación oficial de LiteLLM y en el repositorio diegosouzapw/OmniRoute (rama release/v3.8.52), consultados el 7 de octubre de 2026. Las versiones cambian con frecuencia.

Contenido

1. Qué es un AI Gateway y por qué lo necesitas **3**

Qué hace el gateway en cada petición . . . 3

Qué ganas al usarlo . . . 4

2. LiteLLM u OmniRoute: cuál elegir **4**

Regla práctica . . . 4

3. LiteLLM paso a paso **5**

3.1 Requisitos . . . 5

3.2 Arrancar el gateway con base de datos . . . 5

3.3 Entrar en la Admin UI y añadir un modelo . . . 5

3.4 Crear una clave virtual . . . 5

3.5 Llamar al gateway desde tu aplicación . . . 5

3.6 Configuración declarativa con config.yaml . . . 6

3.7 Balanceo de carga y fallback . . . 7

3.8 Caché semántica . . . 7

3.9 Guardrails: datos personales y prompt injection . . . 8

3.10 Claves, equipos y presupuestos . . . 9

3.11 Qué comprobar y dónde mirar . . . 9

4. OmniRoute paso a paso **10**

4.1 Qué es . . . 10

4.2 Requisitos . . . 10

4.3 Instalación . . . 10

4.4 Primer arranque . . . 11

4.5 Conectar proveedores . . . 11

4.6 Combos y enrutado automático . . . 12

4.7 API key y primera llamada desde tu código . . . 13

4.8 Conectar tus herramientas de código . . . 13

4.9 Resiliencia, caché, compresión y seguridad . . . 14

4.10 Desplegar en un servidor . . . 14

5. Usarlos juntos **15**

6. Lista de comprobación para producción **16**

7. Problemas frecuentes **16**

8. Fuentes **17**

1. Qué es un AI Gateway y por qué lo necesitas

Si tu aplicación llama directamente a OpenAI, Claude o a un modelo propio, cada proveedor impone sus propias claves, costes, límites y reglas de seguridad. Cuando la aplicación crece, esa lógica acaba repartida por todo el código: cambiar de proveedor, repartir presupuesto entre equipos o añadir un filtro de datos personales obliga a tocar muchos sitios.

Un AI Gateway es un servicio que se coloca entre tus aplicaciones y los modelos. Expone un único endpoint, normalmente compatible con la API de OpenAI, y resuelve en un solo lugar la autenticación, la seguridad, el enrutado, la caché, los reintentos y el registro de uso.

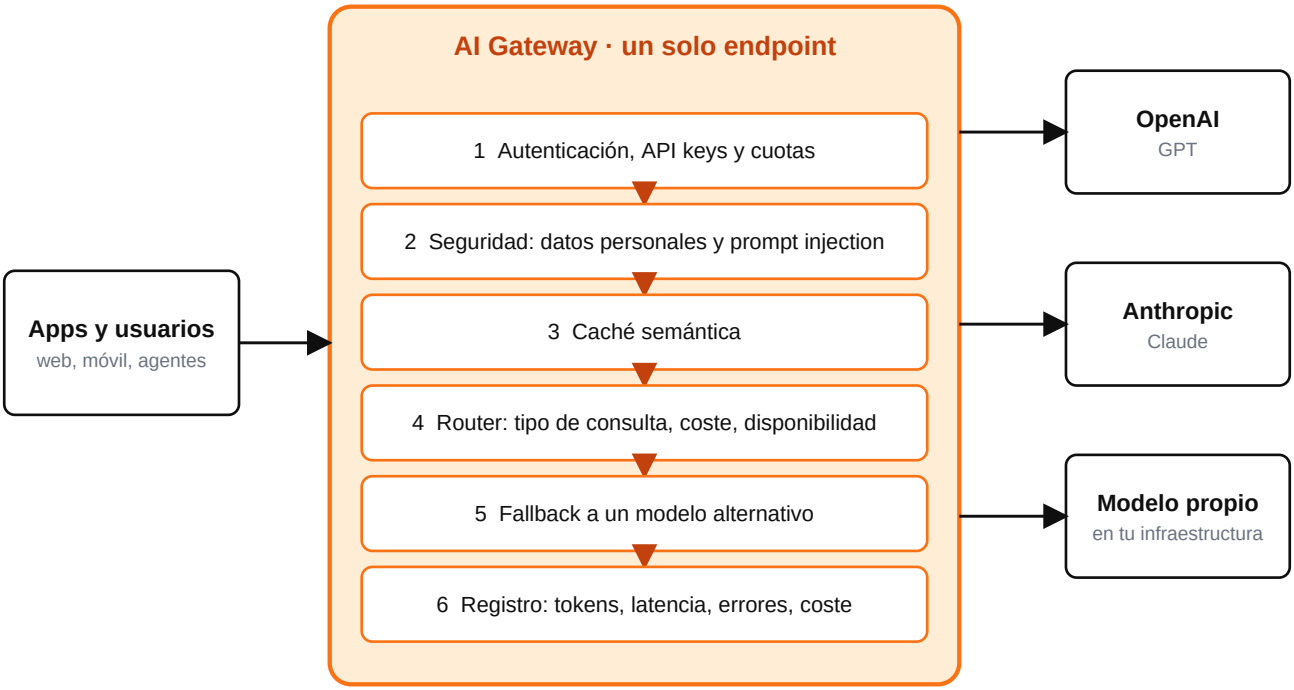


Figura 1. Recorrido de una petición por un AI Gateway. La respuesta vuelve por el mismo camino.

Qué hace el gateway en cada petición

Etapa	Qué hace	Ejemplo
1. Autenticación y cuotas	Comprueba la API key y aplica límites por usuario, equipo o clave.	Una clave con 10 USD de presupuesto diario.
2. Seguridad	Filtra datos personales y detecta inyecciones de prompt antes de llamar al modelo.	Enmascarar un email o una tarjeta en el texto del usuario.
3. Caché semántica	Si ya existe una respuesta a una consulta parecida, la reutiliza.	Dos preguntas casi iguales cuestan una sola llamada al proveedor.
4. Router	Elige el modelo según el tipo de consulta, el coste o la disponibilidad.	Consultas simples a un modelo barato y código a uno más potente.
5. Fallback	Si el modelo principal falla, reenvía la petición a una alternativa.	Un error 429 de un proveedor se resuelve con otro.
6. Registro	Guarda tokens, latencia, errores y coste de cada petición.	Informe de gasto por equipo y por modelo.

La respuesta puede enviarse con streaming, de modo que el usuario ve el texto a medida que se genera y no espera a que termine toda la respuesta.

Qué ganas al usarlo

- La aplicación solo conoce un endpoint y una clave.
- Cambiar de proveedor o añadir un modelo pasa a ser un cambio de configuración y no de código.
- Los controles de seguridad y de coste se aplican igual a todas las aplicaciones.
- Hay una única fuente de datos para auditar uso y gasto.

> **Cuándo no hace falta**
>
> Un prototipo que usa un solo proveedor y una sola clave no necesita gateway. Empieza a compensar cuando hay varios proveedores, varios equipos o requisitos de coste y seguridad.

2. LiteLLM u OmniRoute: cuál elegir

Los dos proyectos son de código abierto con licencia MIT y exponen una API compatible con OpenAI, pero están orientados a usos distintos. La tabla resume las diferencias que importan al decidir.

Aspecto	LiteLLM	OmniRoute
Enfoque	Gateway para aplicaciones y equipos: claves virtuales, presupuestos, equipos y guardrails.	Gateway local-first para herramientas de código y agentes: muchos proveedores, combos y fallback automático.
Puerto por defecto	4000	20128
Instalación	Docker Compose (gateway y PostgreSQL) o contenedor con un config.yaml.	npm, Docker, aplicación de escritorio (Electron) o desde el código fuente.
Estado	PostgreSQL: claves, presupuestos y gasto.	SQLite local.
Interfaz	Admin UI en la ruta /ui.	Dashboard web en el mismo puerto.
Enrutado	Balanceo por model_name, reintentos, fallbacks y auto routing (beta).	19 estrategias de combo y modelos auto/* con puntuación en vivo.
Caché	Exacta (Redis) y semántica (Redis, Qdrant o Valkey).	Caché semántica integrada y compresión de tokens.
Seguridad	Guardrails: Presidio, Lakera, Aporia y otros.	Guard de prompt injection; enmascarado de credenciales y de datos personales opcional.
Proveedores	Claves de API de cada proveedor.	Claves de API, cuentas por OAuth y un catálogo de planes gratuitos.
A tener en cuenta	Presupuestos y claves virtuales requieren base de datos; algunas funciones son Enterprise.	Revisar los términos de cada proveedor. El proyecto cambia con mucha frecuencia.

Regla práctica

- **LiteLLM** si construyes una aplicación o plataforma con varios usuarios o equipos y necesitas claves, presupuestos y trazabilidad.
- **OmniRoute** si quieres dar a Claude Code, Codex, Cursor u otras herramientas un único endpoint con fallback automático y aprovechar cuentas y cuotas gratuitas.
- **Los dos** si necesitas ambas cosas. El capítulo 5 explica cómo encadenarlos.

3. LiteLLM paso a paso

LiteLLM se distribuye como un gateway listo para ejecutar. Esta sección arranca el servicio, añade un modelo, crea una clave virtual y después configura fallback, caché, guardrails y presupuestos.

3.1 Requisitos

- Docker y Docker Compose.
- OpenSSL, para generar las claves del gateway.
- Al menos una clave de API de un proveedor de modelos.

3.2 Arrancar el gateway con base de datos

Los tres comandos descargan el fichero de Compose (que define el gateway y una base de datos PostgreSQL), generan las claves del gateway en un fichero `.env` y levantan los servicios en el puerto 4000.

TERMINAL

```
curl -sSLO https://github.com/BerriAI/litellm/raw/main/docker/docker-compose.quickstart.yml
printf 'LITELLM_MASTER_KEY=sk-%s\nLITELLM_SALT_KEY=sk-%s\n' \
  "$(openssl rand -hex 32)" "$(openssl rand -hex 32)" > .env
docker compose -f docker-compose.quickstart.yml up -d
```

> **Guarda bien el fichero .env**
>
> `LITELLM_MASTER_KEY` es la credencial raíz: autoriza todas las llamadas de gestión y sirve como contraseña de la Admin UI. Trátala como una contraseña de administrador y no la subas al repositorio. `LITELLM_SALT_KEY` cifra las claves de proveedor guardadas y no tiene rotación in situ: si la cambias, las credenciales ya guardadas quedan ilegibles.

3.3 Entrar en la Admin UI y añadir un modelo

1. Abre `http://localhost:4000/ui`. El usuario es `admin` y la contraseña es el valor de `LITELLM_MASTER_KEY`.
2. Ve a **Models + Endpoints** y abre la pestaña **Add Model**.
3. Elige el proveedor, selecciona los modelos del catálogo y pega tu clave de proveedor.
4. Pulsa **Test Connect** para comprobar la clave y después **Add Model**.
5. En **Playground** elige el modelo y envía un mensaje para verificar que todo funciona.

Si prefieres no guardar las claves de proveedor en la interfaz, define variables de entorno en el servicio `litellm` del fichero de Compose (por ejemplo `OPENAI_API_KEY: ${OPENAI_API_KEY}`) y escribe `os.environ/OPENAI_API_KEY` en el campo de la clave.

3.4 Crear una clave virtual

Las claves virtuales son las que entregas a aplicaciones y compañeros en lugar de las claves reales del proveedor. Cada una puede tener su presupuesto, sus límites y su lista de modelos, y todo su gasto queda registrado. En **Virtual Keys** pulsa **+ Create New Key**, ponle un nombre y créala. La clave se muestra una sola vez, así que cópiala en ese momento.

3.5 Llamar al gateway desde tu aplicación

El gateway es compatible con OpenAI. Cualquier SDK de OpenAI funciona si apuntas su URL base a `http://localhost:4000` y usas la clave virtual. Sustituye el nombre del modelo por el que hayas registrado.

CURL

```
curl http://localhost:4000/v1/chat/completions \
  -H 'Authorization: Bearer sk-<tu-clave-virtual>' \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "gpt-5.6-terra",
    "messages": [{"role": "user", "content": "Saluda en cinco palabras."}]
  }'
```

PYTHON (SDK DE OPENAI)

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:4000", api_key="sk-<tu-clave-virtual>")

resp = client.chat.completions.create(
    model="gpt-5.6-terra",
    messages=[{"role": "user", "content": "Saluda en cinco palabras."}],
)
print(resp.choices[0].message.content)
```

3.6 Configuración declarativa con config.yaml

La interfaz es cómoda para empezar. Para versionar la configuración en Git conviene describirla en un `config.yaml`. El ejemplo registra dos proveedores y un modelo propio, define el orden de fallback y los reintentos.

LITELLM_CONFIG.YAML

```
model_list:
  - model_name: chat-principal
    litellm_params:
      model: openai/gpt-5.6-terra
      api_key: os.environ/OPENAI_API_KEY

  - model_name: chat-respaldo
    litellm_params:
      model: anthropic/claude-sonnet-5
      api_key: os.environ/ANTHROPIC_API_KEY

  - model_name: modelo-propio
    litellm_params:
      model: openai/mi-modelo            # servidor compatible con OpenAI (vLLM, Ollama...)
      api_base: http://mi-servidor:8000/v1
      api_key: os.environ/MI_MODELO_API_KEY   # si no exige clave, define cualquier valor

litellm_settings:
  num_retries: 3          # reintentos sobre cada model_name
  request_timeout: 30     # segundos antes de dar la llamada por fallida
  fallbacks: [{"chat-principal": ["chat-respaldo", "modelo-propio"]}]
  allowed_fails: 3        # fallos por minuto antes de enfriar el modelo
  cooldown_time: 30       # segundos de enfriamiento

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
```

Para probarlo sin base de datos, monta el fichero en el contenedor oficial:

TERMINAL

```
docker run \
  -v $(pwd)/litellm_config.yaml:/app/config.yaml \
  -e OPENAI_API_KEY=<tu-clave-openai> \
  -e ANTHROPIC_API_KEY=<tu-clave-anthropic> \
  -e MI_MODELO_API_KEY=<valor> \
  -e LITELLM_MASTER_KEY=sk-<clave-larga-aleatoria> \
  -p 4000:4000 \
  docker.litellm.ai/berriai/litellm:latest \
  --config /app/config.yaml
```

Sin base de datos no hay claves virtuales ni presupuestos

En este modo solo funciona la clave maestra. Las peticiones con una clave virtual fallan con el error `No connected db.` y `max_budget` no limita el gasto: el proxy sigue sirviendo peticiones aunque se supere. Si necesitas presupuestos, usa el modo con PostgreSQL. Para combinar ambos, añade el volumen del `config.yaml` y el parámetro `--config` al servicio `litellm` del Compose descargado, como indica la página Config.yaml de la documentación.

3.7 Balanceo de carga y fallback

El gateway trabaja con grupos de modelos identificados por `model_name`. Si varias entradas comparten el mismo `model_name`, LiteLLM reparte las peticiones entre ellas. Cuando una llamada falla después de `num_retries` intentos, pasa al siguiente grupo de la lista de `fallbacks`, que se recorre en el orden indicado. Hay tres tipos:

- `fallbacks`: para el resto de errores, por ejemplo un 429 por límite de peticiones.
- `context_window_fallbacks`: para prompts que superan la ventana de contexto del modelo.
- `content_policy_fallbacks`: para rechazos por política de contenido.

Para comprobar qué despliegue atendió una petición, mira la cabecera de respuesta `x-litellm-model-id`. Las banderas `mock_testing_fallbacks` ya no tienen efecto en el proxy a partir de la versión 1.85.0, así que la prueba debe hacerse provocando un error real (por ejemplo, con una clave inválida) en un entorno que no sea de producción.

3.8 Caché semántica

Una caché semántica convierte el prompt en un embedding y devuelve la respuesta guardada más cercana si supera un umbral de similitud. Por eso puede acertar con preguntas parecidas y no solo con idénticas. Necesita un modelo de embeddings registrado en `model_list` y un backend vectorial (Redis con búsqueda vectorial, Qdrant o Valkey con el módulo valkey-search).

FRAGMENTO DE CONFIG.YAML

```
model_list:
  - model_name: openai-embedding
    litellm_params:
      model: openai/text-embedding-3-small
      api_key: os.environ/OPENAI_API_KEY

litellm_settings:
  cache: True
  cache_params:
    type: "redis-semantic"
    similarity_threshold: 0.8
    redis_semantic_cache_embedding_model: openai-embedding
```

Configura las credenciales de Redis como indica la página de Redis y Valkey de la documentación (variables `REDIS_HOST`, `REDIS_PORT` y `REDIS_PASSWORD`). Cuando una respuesta sale de la caché semántica, la petición

devuelve la cabecera `x-litellm-semantic-similarity`.

> **No la actives en tráfico de agentes**
>
> Los agentes de código reenvían toda la conversación en cada turno, así que dos turnos consecutivos son casi idénticos y la caché devuelve la respuesta anterior. El síntoma típico es un agente que repite la misma llamada a una herramienta. La documentación recomienda excluir ese tráfico, por ejemplo con `"cache": {"no-cache": true}` en los metadatos de las claves virtuales de los agentes, o usar la caché exacta (`type: redis`) en su lugar.

3.9 Guardrails: datos personales y prompt injection

Los guardrails se declaran en la sección `guardrails` y se ejecutan en momentos distintos de la petición según el valor de `mode`.

mode	Cuándo se ejecuta
pre_call	Antes de llamar al modelo, sobre la entrada.
during_call	En paralelo a la llamada al modelo, sobre la entrada. La respuesta espera al resultado.
post_call	Después de la llamada, sobre entrada y salida.
logging_only	Analiza lo que se registra sin cambiar la respuesta al cliente.

ENMASCARAR DATOS PERSONALES CON PRESIDIO

```
guardrails:
  - guardrail_name: "presidio-pii"
    litellm_params:
      guardrail: presidio
      mode: "pre_call"
      presidio_language: "en"
      default_on: true            # se aplica a todas las peticiones
      pii_entities_config:
        CREDIT_CARD: "MASK"
        EMAIL_ADDRESS: "MASK"
      presidio_score_thresholds:
        CREDIT_CARD: 0.8
        EMAIL_ADDRESS: 0.6
```

Presidio necesita sus servicios de análisis y anonimización accesibles desde el gateway; consulta la página del guardrail para las variables concretas. Los reconocedores dependen del idioma, así que ajusta `presidio_language` y las entidades a tus datos. Para detectar inyección de prompts, la documentación lista proveedores como Lakera y Aporia, que se declaran con el mismo formato:

PROVEEDOR EXTERNO

```
  - guardrail_name: "aporia-pre-guard"
    litellm_params:
      guardrail: aporia          # valores admitidos: "aporia", "lakera"
      mode: "pre_call"
      api_key: os.environ/APORIA_API_KEY_1
      api_base: os.environ/APORIA_API_BASE_1
```

Puedes activar guardrails concretos por petición con el campo `"guardrails": ["nombre"]` y comprobar cuáles se aplicaron en la cabecera `x-litellm-applied-guardrails`. Asignar guardrails por clave o por equipo es una función Enterprise.

3.10 Claves, equipos y presupuestos

Estas llamadas usan la clave maestra y requieren el modo con base de datos. El primer ejemplo crea una clave con dos ventanas de presupuesto independientes: 10 USD al día y 100 USD cada 30 días.

CLAVE CON PRESUPUESTO DIARIO Y MENSUAL

```
curl 'http://localhost:4000/key/generate' \
  --header 'Authorization: Bearer <tu-master-key>' \
  --header 'Content-Type: application/json' \
  --data-raw '{
    "budget_limits": [
      {"budget_duration": "24h", "max_budget": 10},
      {"budget_duration": "30d", "max_budget": 100}
    ]
  }'
```

EQUIPO CON LÍMITE DE PETICIONES POR MINUTO

```
curl 'http://localhost:4000/team/new' \
  --header 'Authorization: Bearer <tu-master-key>' \
  --header 'Content-Type: application/json' \
  --data-raw '{"team_alias": "equipo-producto", "rpm_limit": 99}'
```

- Si una clave pertenece a un equipo, solo se aplican los presupuestos del equipo y de sus miembros; el presupuesto personal del propietario de la clave no cuenta.
- Los presupuestos por modelo son una función Enterprise.
- Con `budget_duration` el gasto se reinicia al terminar el periodo. Admite segundos, minutos, horas y días (por ejemplo `30d`).

3.11 Qué comprobar y dónde mirar

Cabecera o pantalla	Qué te dice
x-litellm-model-id	Qué despliegue atendió la petición (útil para verificar un fallback).
x-litellm-applied-guardrails	Qué guardrails se ejecutaron.
x-litellm-semantic-similarity	La petición se sirvió desde la caché semántica.
x-litellm-key-remaining-requests-{modelo}	Peticiones que le quedan a la clave para ese modelo.
Admin UI	Modelos, claves, equipos, gasto y Playground.

Para producción, la documentación tiene guías de despliegue con Helm, Terraform y Kubernetes, y una lista de buenas prácticas con Redis para varias instancias. Fija una versión concreta de la imagen en lugar de `latest` editando el fichero de Compose.

4. OmniRoute paso a paso

4.1 Qué es

OmniRoute es un gateway de IA de código abierto (licencia MIT) que ofrece un único endpoint compatible con OpenAI en `http://localhost:20128/v1` y reparte las peticiones entre cientos de proveedores. Según su README, el catálogo supera los 350 proveedores, unos 150 con algún plan gratuito, y funciona con Claude Code, Codex, Cursor, Cline, OpenCode y cualquier herramienta compatible con OpenAI. Esas cifras las publica y actualiza el propio proyecto, así que trátalas como orientativas.

> **Revisa los términos de cada proveedor**
>
> Conectar una cuenta de suscripción por OAuth o usar planes gratuitos está sujeto a las condiciones de cada proveedor. OmniRoute mantiene un catálogo que marca los proveedores con riesgo en sus términos (13 en la revisión actual del README). Consúltalo antes de conectar una cuenta y decide con ese dato.

4.2 Requisitos

- Node.js 22.x o 24.x LTS (el rango indicado es `>=22.22.2 <23 || >=24.0.0 <27`), o Docker.
- El puerto 20128 libre.

4.3 Instalación

La vía más rápida es npm. Docker es preferible en un servidor.

NPM (GLOBAL)

```
npm install -g omniroute
omniroute
```

DOCKER

```
docker run -d --name omniroute --restart unless-stopped --stop-timeout 40 \
  -p 127.0.0.1:20128:20128 -v omniroute-data:/app/data diegosouzapw/omniroute:latest
```

La etiqueta `:latest` sigue la última versión estable publicada. Para despliegues reproducibles fija una versión con `:X.Y.Z`. La imagen limita la memoria de Node a 1 GB, suficiente para el dashboard y un chat ligero. Si vas a conectar agentes de código, amplía el heap y el límite del contenedor:

Carga	Heap (OMNIROUTE_MEMORY_MB)	Contenedor (--memory)
Dashboard o chat ligero	1024 (valor de la imagen)	2 GB o más
Un agente de código	8192	10 GB o más
Dos peticiones largas simultáneas a /v1/responses	10240 a 12288	12 a 16 GB

DOCKER PARA AGENTES DE CÓDIGO

```
docker run -d --name omniroute --restart unless-stopped --stop-timeout 40 \
  -e OMNIROUTE_MEMORY_MB=8192 --memory=10g \
  -p 127.0.0.1:20128:20128 -v omniroute-data:/app/data diegosouzapw/omniroute:latest
```

Otras opciones que documenta el proyecto:

Método	Comando
pnpm	`pnpm add -g omniroute@latest --allow-build=better-sqlite3 --allow-build=@swc/core && omniroute`
Desde el código fuente	`cp .env.example .env && npm install` y después `PORT=20128 npm run dev`
Android (Termux)	`pkg install nodejs && npx -y omniroute`
Arch Linux (AUR)	`yay -S omniroute-bin && systemctl --user enable --now omniroute.service`
Escritorio (Electron)	`npm run electron:build`, con instaladores para Windows, macOS y Linux
Podman	Guía propia en contrib/podman/README.md del repositorio

Si npm muestra avisos ERESOLVE o de dependencias entre pares, su guía de problemas los considera inofensivos.

4.4 Primer arranque

Tras ejecutar `omniroute`, el dashboard queda en `http://localhost:20128` y la API en `http://localhost:20128/v1`. El comando `omniroute doctor` diagnostica proveedores, puertos y dependencias nativas, y `omniroute setup` lanza un asistente de primera configuración.

Una instalación nueva ya responde sin claves ni configuración, porque incluye un proveedor sin autenticación (OpenCode Free) integrado en el modelo `auto`:

PRUEBA SIN CREDENCIALES

```
curl http://localhost:20128/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"auto","messages":[{"role":"user","content":"Hola"}]}'
```

4.5 Conectar proveedores

En el dashboard abre **Providers** y conecta los que quieras usar. Hay tres tipos de conexión:

- **Gratuitos.** OpenCode Free no requiere autenticación. Kiro AI se conecta con AWS Builder ID o con Google o GitHub y ofrece unos 50 créditos al mes por cuenta.
- **Con clave de API.** Elige **Add API Key**, indica el proveedor (por ejemplo `glm` o `groq`) y pega la clave.
- **Con OAuth.** Claude Code, Codex y GitHub Copilot se conectan iniciando sesión; revisa antes la advertencia de términos del apartado 4.1.

Los modelos se nombran con el formato `prefijo/modelo`. Algunos prefijos que usa la guía oficial:

Prefijo	Proveedor	Prefijo	Proveedor
cc/	Claude Code (OAuth)	glm/	GLM (Zhipu)
cx/	Codex (OAuth)	minimax/	MiniMax
gh/	GitHub Copilot (OAuth)	kimi/	Kimi
kr/	Kiro (gratuito)	ds/	DeepSeek
oc/	OpenCode Free	groq/	Groq

Los identificadores de modelo cambian con las versiones. Para ver los vigentes usa `omniroute models`, la sección **Available Models** de cada proveedor o `GET /api/models/catalog`.

4.6 Combos y enrutado automático

Un **combo** es una cadena de modelos entre los que OmniRoute enruta. Si se agota la cuota, falla un proveedor o sube el coste, pasa al siguiente modelo sano. Se crea en **Dashboard > Combos > Create New**: pones un nombre, añades los modelos en orden y eliges la estrategia. Después usas el nombre del combo como valor de `model` en cualquier cliente.

EJEMPLO DE COMBO (COMPRUEBA LOS IDS VIGENTES)

```
Combo: codigo-diario        Estrategia: priority
  1. cc/claude-sonnet-4-6      (suscripción: se usa primero)
  2. glm/glm-4.7               (de pago y barato: respaldo)
  3. kr/claude-haiku-4.5       (gratuito: último recurso)
```

OmniRoute implementa 19 estrategias. Estas son las más utilizadas:

Estrategia	Qué hace
priority	Usa el primer destino de la lista y pasa al siguiente solo si falla.
weighted	Reparte de forma aleatoria según el peso de cada destino.
round-robin	Rota por los destinos en orden.
p2c	Elige dos al azar y envía al más sano.
least-used	Envía al de menor carga actual.
cost-optimized	Minimiza el coste por petición con los precios del catálogo.
lkgp	Se queda con el último proveedor que funcionó y recurre a las reglas si falla.
fusion y pipeline	Fusion consulta a varios modelos y un juez sintetiza la respuesta; pipeline encadena pasos donde la salida de uno alimenta al siguiente.

Si no quieres mantener combos, usa los modelos virtuales `auto`, que puntúan en vivo a los proveedores conectados y construyen el combo en cada petición:

Modelo	Optimiza
auto	Equilibrio general; se queda con el último proveedor que funcionó.
auto/coding	Calidad para generación de código.
auto/fast	Menor latencia.
auto/cheap	Menor coste por token.
auto/offline	Mayor margen de cuota y de límites.
auto/smart	Calidad primero, con un 10% de exploración de modelos nuevos.
auto/lkgp	Fidelidad al último proveedor correcto.

También puedes gestionar combos por API con una clave de ámbito *manage* (**Dashboard > API Keys > New Key**):

API DE GESTIÓN

```
curl -X POST http://localhost:20128/api/combos \
  -H "Authorization: Bearer $OMNIROUTE_MANAGE_KEY" \
  -H "Content-Type: application/json" \
  -d '{ "name": "premium", "strategy": "priority",
        "models": [{ "model": "cc/claude-sonnet-4-6" }, { "model": "glm/glm-4.7" }] }'
```

4.7 API key y primera llamada desde tu código

Copia la clave en **Dashboard > Endpoints** y comprueba que el gateway lista tus modelos:

LISTAR MODELOS

```
curl http://localhost:20128/v1/models -H "Authorization: Bearer TU_CLAVE"
```

PYTHON (SDK DE OPENAI)

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:20128/v1", api_key="TU_CLAVE_OMNIROUTE")

resp = client.chat.completions.create(
    model="auto/coding",          # o el nombre de uno de tus combos
    messages=[{"role": "user", "content": "Refactoriza esta función de Python"}],
    stream=True,
)
for chunk in resp:
    print(chunk.choices[0].delta.content or "", end="")
```

En local el gateway responde sin clave. En un servidor define `REQUIRE_API_KEY=true` para exigir una clave Bearer en todas las rutas `/v1/*`. Si un cliente no puede enviar cabeceras, OmniRoute ofrece alias con la clave dentro de la URL (`/vscode/TU_CLAVE/`); úsalos solo en ese caso, porque la clave puede quedar registrada en logs.

4.8 Conectar tus herramientas de código

Claude Code. Edita `~/.claude/settings.json`. La URL es la raíz del gateway, sin `/v1`.

~/.CLAUDE/SETTINGS.JSON

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:20128",
    "ANTHROPIC_AUTH_TOKEN": "tu-clave-omniroute"
  }
}
```

Codex CLI.

TERMINAL

```
export OPENAI_BASE_URL="http://localhost:20128"
export OPENAI_API_KEY="tu-clave-omniroute"
codex "tu prompt"
```

Cursor, Cline, Continue y RooCode. Elige un proveedor compatible con OpenAI y usa `http://localhost:20128/v1` como URL base, la clave del dashboard y un modelo, un combo o `auto`.

Lanzador de una línea. OmniRoute inyecta las credenciales solo en ese proceso y no escribe ficheros de configuración. Con `--dry-run` ves el entorno y los argumentos sin ejecutar nada.

TERMINAL

```
omniroute run claude --model openai/gpt-5.4
omniroute run codex  --model glm/glm-5.2
omniroute configure codex       # elige proveedor y modelo y escribe el perfil de la herramienta
```

4.9 Resiliencia, caché, compresión y seguridad

Área	Cómo funciona
Resiliencia en 3 capas	1) Circuit breaker del proveedor, que solo se dispara con errores 408 y 5xx. 2) Enfriamiento de una conexión o cuenta, que respeta Retry-After en los 429. 3) Bloqueo de un modelo concreto sin afectar al resto de la conexión. Las claves del mismo proveedor siguen atendiendo mientras una se enfría.
Caché semántica	Guarda automáticamente las respuestas sin streaming con temperature=0. Se omite con la cabecera X-OmniRoute-No-Cache: true.
Compresión de tokens	Pipeline de 12 motores que actúa sobre el prompt de forma transparente. El modo apilado RTK + Caveman declara un ahorro del 15 al 95% según la carga. El código, las URL y el JSON se conservan intactos. Se controla por petición con la cabecera x-omniroute-compression.
Seguridad	Credenciales cifradas con AES-256-GCM, telemetría desactivada por defecto, guard de prompt injection en las rutas LLM y, de forma opcional, enmascarado de credenciales y redacción de datos personales. Hay filtrado por IP y ámbitos por clave.
Transparencia	Cada respuesta incluye la cabecera X-OmniRoute-Decision con la estrategia, el proveedor y la latencia.
Presupuestos	En Dashboard > Costs se fijan límites diarios, semanales o mensuales por API key. Cada petición registra tokens y coste.
Observabilidad	Dashboard > Health muestra breakers, cooldowns y latencias p50, p95 y p99. Los webhooks emiten eventos como request.failed, budget.exceeded o circuit_breaker.opened, firmados con HMAC-SHA256.

4.10 Desplegar en un servidor

Las variables por defecto del proyecto son públicas (por ejemplo, la contraseña inicial `CHANGEME`), así que cambia al menos las tres primeras antes de exponer el servicio.

Variable	Para qué sirve
JWT_SECRET	Secreto de firma de sesiones. Cambiarlo es obligatorio en producción.
INITIAL_PASSWORD	Contraseña del primer inicio de sesión en el dashboard.
API_KEY_SECRET	Secreto HMAC con el que se generan las API keys.
REQUIRE_API_KEY	Con true, exige clave Bearer en /v1/*.
AUTH_COOKIE_SECURE	Con true, fuerza cookie Secure; úsalo detrás de un proxy HTTPS.
DATA_DIR	Directorio de datos: base de datos, uso y logs. Por defecto ~/.omniroute.
PORT / HOSTNAME	Puerto (20128 en los ejemplos) y dirección de escucha.
NEXT_PUBLIC_BASE_URL	URL pública que ve el dashboard.
OMNIROUTE_MEMORY_MB	Límite de memoria de Node (512 por defecto fuera de Docker).

DOCKER CON VARIABLES PROPIAS

```
# .env con valores propios (no uses los de ejemplo)
JWT_SECRET=<cadena-larga-aleatoria>
INITIAL_PASSWORD=<contraseña-fuerte>
API_KEY_SECRET=<otra-cadena-aleatoria>
REQUIRE_API_KEY=true
AUTH_COOKIE_SECURE=true

docker run -d --name omniroute --restart unless-stopped --stop-timeout 40 \
  --env-file ./.env -p 127.0.0.1:20128:20128 -v omniroute-data:/app/data \
  diegosouzapw/omniroute:latest
```

- Publica el puerto solo en `127.0.0.1` y pon delante un proxy inverso con HTTPS (Caddy o Nginx). Si usas Nginx y envías cabeceras con guión bajo, activa `underscores_in_headers on`.
- Haz copias de la base de datos con `omniroute backup` o desde **Settings > System & Storage**. Todo el estado está en SQLite, así que no montes la misma base de datos en dos instancias a la vez.
- Hay guías del proyecto para Docker Compose, máquinas virtuales, Fly.io, Termux y Podman en la carpeta `docs/` del repositorio.

5. Usarlos juntos

Es un patrón válido cuando necesitas la gestión de equipos, claves y presupuestos de LiteLLM y, además, el catálogo amplio y los combos de OmniRoute. LiteLLM trata a OmniRoute como un proveedor compatible con OpenAI.

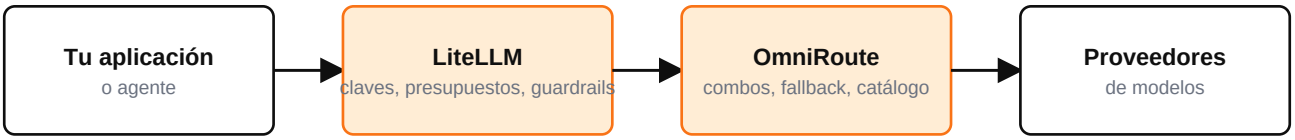


Figura 2. LiteLLM como capa de control y OmniRoute como capa de enrutado.

FRAGMENTO DE CONFIG.YAML DE LITELLM

```
model_list:
  - model_name: coding-auto
    litellm_params:
      model: openai/auto                  # el modelo "auto" de OmniRoute
      api_base: http://omniroute:20128/v1
      api_key: os.environ/OMNIROUTE_API_KEY
```

Antes de adoptarlo

Este montaje añade un salto de red y dos capas de fallback y de caché que pueden solaparse. Pruébalo con tráfico real y deja cada función en una sola capa (por ejemplo, la caché semántica solo en LiteLLM y el fallback entre proveedores solo en OmniRoute). Es una recomendación de diseño, no una configuración documentada por ninguno de los dos proyectos.

6. Lista de comprobación para producción

Área	Comprobación
Acceso	Contraseñas y secretos propios en LiteLLM y OmniRoute. Puerto publicado solo en localhost y HTTPS delante. La clave maestra y las claves de gestión no salen del servidor.
Claves	Una clave virtual o API key por aplicación y por equipo, nunca la clave real del proveedor. Claves con caducidad o presupuesto cuando sea posible.
Coste	Presupuestos y límites de peticiones por clave o por equipo. Alertas de gasto. Modelos baratos como primera opción en el enrutado.
Seguridad	Filtro de datos personales y detección de prompt injection activos por defecto. Revisados los términos de cada proveedor conectado.
Fiabilidad	Al menos un fallback de otro proveedor para cada modelo crítico. Reintentos y tiempos de espera ajustados. Fallback probado con un error real.
Caché	Caché semántica solo en tráfico de una sola consulta; excluida en agentes.
Operación	Versión de la imagen fijada. Copias de la base de datos programadas. Registros y métricas enviados a la herramienta que uses.

7. Problemas frecuentes

Síntoma	Causa probable	Solución
LiteLLM no arranca.	Falta LITELLM_MASTER_KEY.	Defínela en el .env o en el entorno del contenedor.
Las credenciales guardadas dejan de funcionar.	Cambió LITELLM_SALT_KEY.	Restaura el valor original; no tiene rotación in situ.
Error "No connected db." con una clave virtual.	El gateway corre sin base de datos.	Usa el modo con PostgreSQL.
El presupuesto no corta el gasto.	Sin base de datos, max_budget no se aplica.	Ejecuta con base de datos y usa claves o equipos con presupuesto.
Un agente repite la misma respuesta o herramienta.	Caché semántica sobre conversaciones casi idénticas.	Excluye esas claves con no-cache o usa caché exacta.
Claude Code no conecta con OmniRoute.	ANTHROPIC_BASE_URL termina en /v1.	Usa http://localhost:20128 sin /v1.
OmniRoute se cierra con FATAL ERROR en Docker.	Heap de Node insuficiente con agentes de código.	OMNIROUTE_MEMORY_MB=8192 y --memory=10g.
Error 508 Loop Detected.	Dos gateways se llaman entre sí.	Revisa OMNIROUTE_INSTANCE_ID y OMNIROUTE_PEER_URLS.
Un modelo ya no existe.	El proveedor cambió su catálogo.	Consulta omniroute models o la sección Available Models y actualiza el combo.

8. Fuentes

Los comandos y las configuraciones de esta guía proceden de estas páginas, consultadas el 7 de octubre de 2026. No se ejecutaron en un entorno propio, así que prueba cada paso en un entorno de desarrollo y contrasta con la documentación vigente, porque ambos proyectos publican versiones con mucha frecuencia.

LiteLLM, Quickstart
https://docs.litellm.ai/docs/proxy/docker_quick_start

LiteLLM, Fallbacks
https://docs.litellm.ai/docs/proxy/reliability

LiteLLM, Semantic Caching
https://docs.litellm.ai/docs/proxy/caching_semantic

LiteLLM, Guardrails
https://docs.litellm.ai/docs/proxy/guardrails/quick_start

LiteLLM, Budgets y rate limits
https://docs.litellm.ai/docs/proxy/users

LiteLLM, Routing y load balancing
https://docs.litellm.ai/docs/routing-load-balancing

OmniRoute, repositorio y README
https://github.com/diegosouzapw/OmniRoute

OmniRoute, User Guide
https://github.com/diegosouzapw/OmniRoute/blob/main/docs/guides/USER_GUIDE.md